Overview of Statistical Learning and Modeling – 36-600

Lab 3R: Exploratory Data Analysis
Due Friday, September 12, 11:59pm

Trishla Nair

In today's lab, you will perform exploratory data analysis on a dataset related to heart disease and the cost billed to insurance.

Data

Your first job is to retrieve the dataset, `heart_disease.csv`, from the same lab module page you found this assignment.

Examine the downloaded data file. Think about how you would input these data. *Hints*: do any strings represent factor variables? Do you need to specify column types? And so on... then:

Questions

Question 1

Read the data into `R`, specifically into a data frame named `df`.

```
df<-read.csv("heart_disease.csv")
head(df)
```

```
##      Cost Age Gender Interventions Drugs ERVisit Complications Comorbidities
## 1   179.1  63 Female             2     1       4             0             3
## 2   319.0  59 Female             2     0       6             0             0
## 3  9310.7  62 Female            17     0       2             0             5
## 4   280.9  60   Male             9     0       7             0             2
## 5 18727.1  55 Female             5     2       7             0             0
## 6   453.4  66 Female             1     0       3             0             4
##   Duration id
## 1      300  1
## 2      120  2
## 3      353  3
## 4      332  4
## 5       18  5
## 6      296  6
```

Question 2

Summarize the data, via a base-R function mentioned in today's notes. Scan the output to see if there are missing data or if anything appears odd.

```
summary(df)
```

```
##       Cost              Age           Gender           Interventions
##  Min.   :    0.0   Min.   :24.00   Length:788         Min.   : 0.000
##  1st Qu.:  161.1   1st Qu.:55.00   Class :character   1st Qu.: 1.000
##  Median :  507.2   Median :60.00   Mode  :character   Median : 3.000
##  Mean   : 2800.0   Mean   :58.72                      Mean   : 4.707
##  3rd Qu.: 1905.5   3rd Qu.:64.00                      3rd Qu.: 6.000
##  Max.   :52664.9   Max.   :70.00                      Max.   :47.000
##      Drugs            ERVisit       Complications     Comorbidities
##  Min.   :0.0000   Min.   : 0.000   Min.   :0.00000   Min.   : 0.000
##  1st Qu.:0.0000   1st Qu.: 2.000   1st Qu.:0.00000   1st Qu.: 0.000
##  Median :0.0000   Median : 3.000   Median :0.00000   Median : 1.000
##  Mean   :0.3388   Mean   : 3.425   Mean   :0.05457   Mean   : 3.767
##  3rd Qu.:0.0000   3rd Qu.: 5.000   3rd Qu.:0.00000   3rd Qu.: 5.000
##  Max.   :2.0000   Max.   :20.000   Max.   :1.00000   Max.   :60.000
##     Duration            id
##  Min.   :  0.00   Min.   :  1.0
##  1st Qu.: 41.75   1st Qu.:197.8
##  Median :165.50   Median :394.5
##  Mean   :164.03   Mean   :394.5
##  3rd Qu.:281.00   3rd Qu.:591.2
##  Max.   :372.00   Max.   :788.0
```

Question 3

One thing you might have noticed in Question 2 is that `Drugs` apparently can only take on the values 0, 1, and 2, and that `Complications` is only either 0 or 1. This hints that these are actually factor variables, and not numeric! For purposes of visualization and analysis, it can be helpful to forcibly transform these variables from `numeric` type to `factor` type. You would do that as follows:

```
df$Drugs <- factor(df$Drugs)
```

Convert both variables, and re-display the summary.

```
df$Drugs <- factor(df$Drugs)
df$Complications <- factor(df$Complications)
summary(df)
```

```
##       Cost              Age           Gender           Interventions    Drugs
##  Min.   :    0.0   Min.   :24.00   Length:788         Min.   : 0.000   0:610
##  1st Qu.:  161.1   1st Qu.:55.00   Class :character   1st Qu.: 1.000   1: 89
##  Median :  507.2   Median :60.00   Mode  :character   Median : 3.000   2: 89
##  Mean   : 2800.0   Mean   :58.72                      Mean   : 4.707
##  3rd Qu.: 1905.5   3rd Qu.:64.00                      3rd Qu.: 6.000
##  Max.   :52664.9   Max.   :70.00                      Max.   :47.000
```

```
##     ERVisit       Complications Comorbidities       Duration
##  Min.   : 0.000   0:745         Min.   : 0.000   Min.   :  0.00
##  1st Qu.: 2.000   1: 43         1st Qu.: 0.000   1st Qu.: 41.75
##  Median : 3.000                 Median : 1.000   Median :165.50
##  Mean   : 3.425                 Mean   : 3.767   Mean   :164.03
##  3rd Qu.: 5.000                 3rd Qu.: 5.000   3rd Qu.:281.00
##  Max.   :20.000                 Max.   :60.000   Max.   :372.00
##        id
##  Min.   :  1.0
##  1st Qu.:197.8
##  Median :394.5
##  Mean   :394.5
##  3rd Qu.:591.2
##  Max.   :788.0
```

Question 4

Look at your summary output again. Are there any obviously non-informative columns? If so, remove them here. For instance, use `dplyr` functions to remove the offending column(s), and save the output to `df`. *Recall*: to remove a single column, you can name it and put a minus sign in front of it in a `dplyr` function call. Then show the names of the columns of `df` to verify that the offending column(s) are gone.

```
suppressMessages(library(tidyverse))

df<- df%>%select(-id)
summary(df)
```

```
##       Cost              Age           Gender           Interventions    Drugs
##  Min.   :    0.0   Min.   :24.00   Length:788         Min.   : 0.000   0:610
##  1st Qu.:  161.1   1st Qu.:55.00   Class :character   1st Qu.: 1.000   1: 89
##  Median :  507.2   Median :60.00   Mode  :character   Median : 3.000   2: 89
##  Mean   : 2800.0   Mean   :58.72                      Mean   : 4.707
##  3rd Qu.: 1905.5   3rd Qu.:64.00                      3rd Qu.: 6.000
##  Max.   :52664.9   Max.   :70.00                      Max.   :47.000
##     ERVisit       Complications Comorbidities       Duration
##  Min.   : 0.000   0:745         Min.   : 0.000   Min.   :  0.00
##  1st Qu.: 2.000   1: 43         1st Qu.: 0.000   1st Qu.: 41.75
##  Median : 3.000                 Median : 1.000   Median :165.50
##  Mean   : 3.425                 Mean   : 3.767   Mean   :164.03
##  3rd Qu.: 5.000                 3rd Qu.: 5.000   3rd Qu.:281.00
##  Max.   :20.000                 Max.   :60.000   Max.   :372.00
```

When performing exploratory data analysis and reporting the results to others, it can be quite helpful to employ "faceting" to save space. (It makes the plots smaller, but it also makes them easier to compare against each other.)

In Question 5 below, you will create faceted plots for all the truly quantitative variables (meaning, everything but `Gender`, `Drugs`, and `Complications`). The typical approach is to use histograms. But, how do you create the plots?

First, you have to `select()` the columns of data you wish to visualize (while selecting either "positively" or "negatively" – generally whichever leads to less typing!), to pass the selected columns to the `gather()` function, and save the output from `gather()` (say, as `df.quant`).

What `gather()` does is output a data frame whose first column, `key`, is a factor variable with column names, and whose second column, `value`, contains the variable values. For instance, it turns

```
a   b   c
1   2   3
4   5   6
```

into

```
key   value
a     1
a     4
b     2
b     5
c     3
c     6
```

The next step would be to take the output from `gather()` (saved here as `df.quant`) and make a histogram of it, in the same manner as in the lecture notes, *but* we'll need to add one additional line:

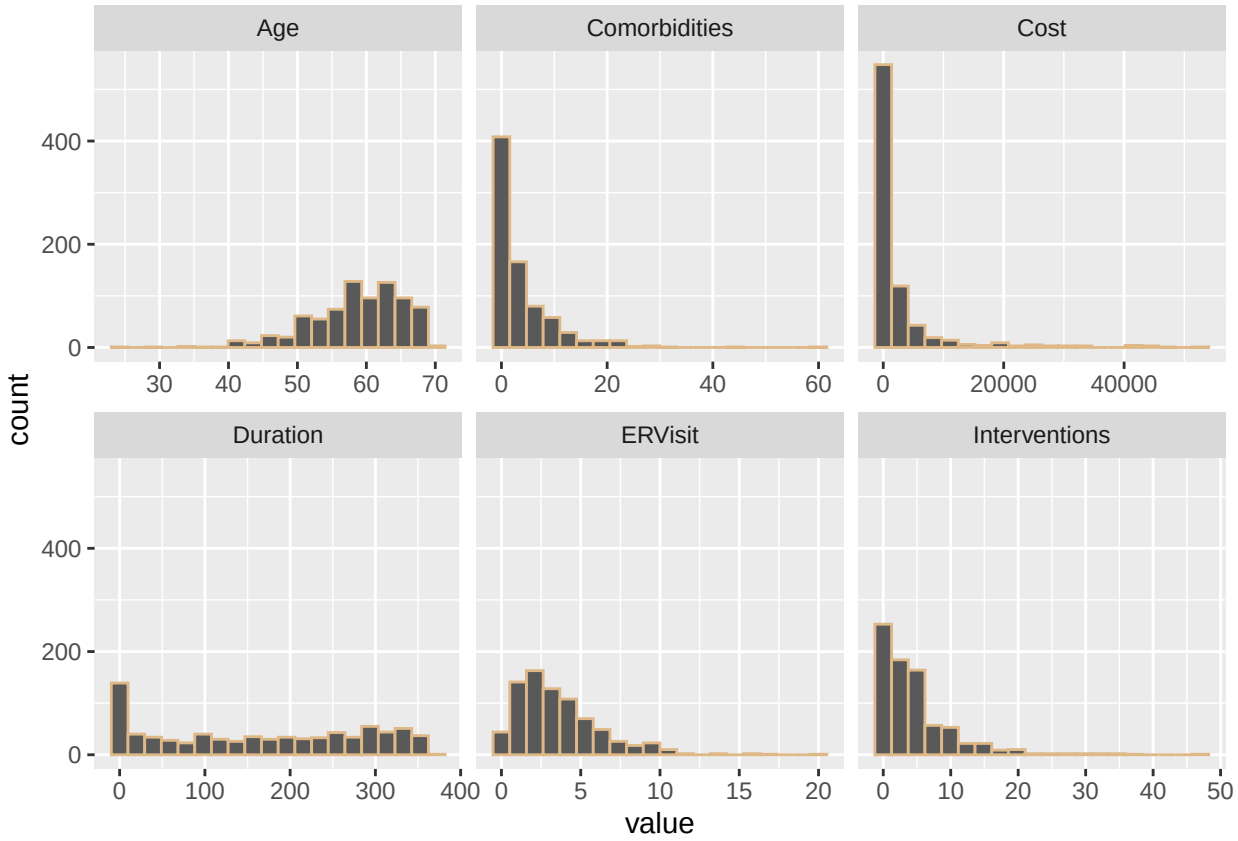
```
  ... +
  geom_histogram(col = "burlywood") +   # or whatever color, bin size, etc. we prefer
  facet_wrap(~key, scales = 'free_x')
```

About the `scales` argument: `free_x` means "let the plot limits along the x-axis differ in each histogram but keep the y-axis limits fixed," `free_y` means the opposite, and `free` means "let the plot limits along each axis vary at will." Play with this to see which allows for the best pictorial representation of the distributions of variable values.

Question 5

Create a faceted histogram for all the variables that are truly quantitative, meaning leave `Gender`, `Drugs`, and `Complications` out. Try `scales = "free_x"` and `scales = "free"` and keep the one that leads to "better" results. (Note that when constructing the histograms, the `x` variable in the call to `aes()` is `value`.)

```
selected<-df%>%select(-Gender, -Drugs, - Complications)
df.quant <-selected%>%gather(.)
ggplot(data = df.quant, mapping = aes(x = value)) +
  geom_histogram(col = "burlywood", bins = 20) +
  facet_wrap(~key, scales = 'free_x')
```



Question 6

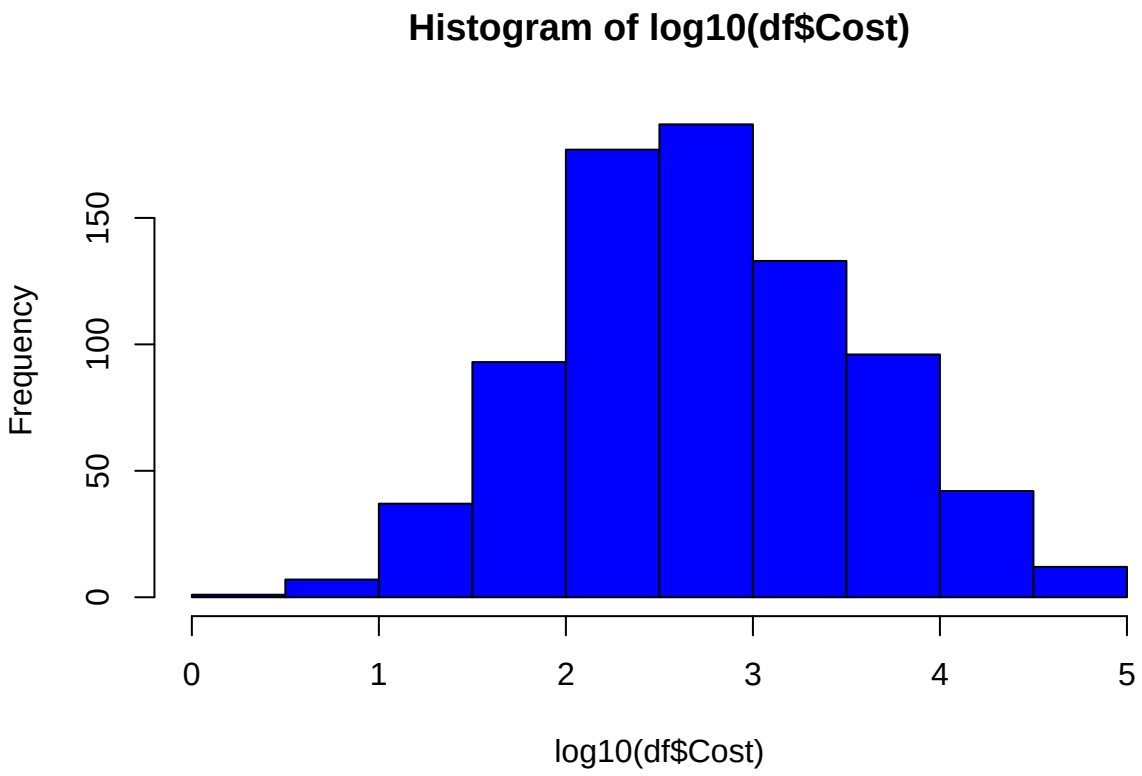
Look at `Cost`: it is skewed right. Make a histogram of the base-10 logarithm of `Cost`, i.e., do:

```
hist(log10(df$Cost), col = "blue")   # quick'n'dirty plotting, no ggplot needed here!
```

Does this look more symmetric? If yes, replace the `Cost` column, i.e., do

```
df %>% filter(., Cost > 0) -> df
df$Cost <- log10(df$Cost)
```

```
hist(log10(df$Cost), col = "blue")
```




```
df %>% filter(., Cost > 0) -> df
df$Cost <- log10(df$Cost)
```

Question 7

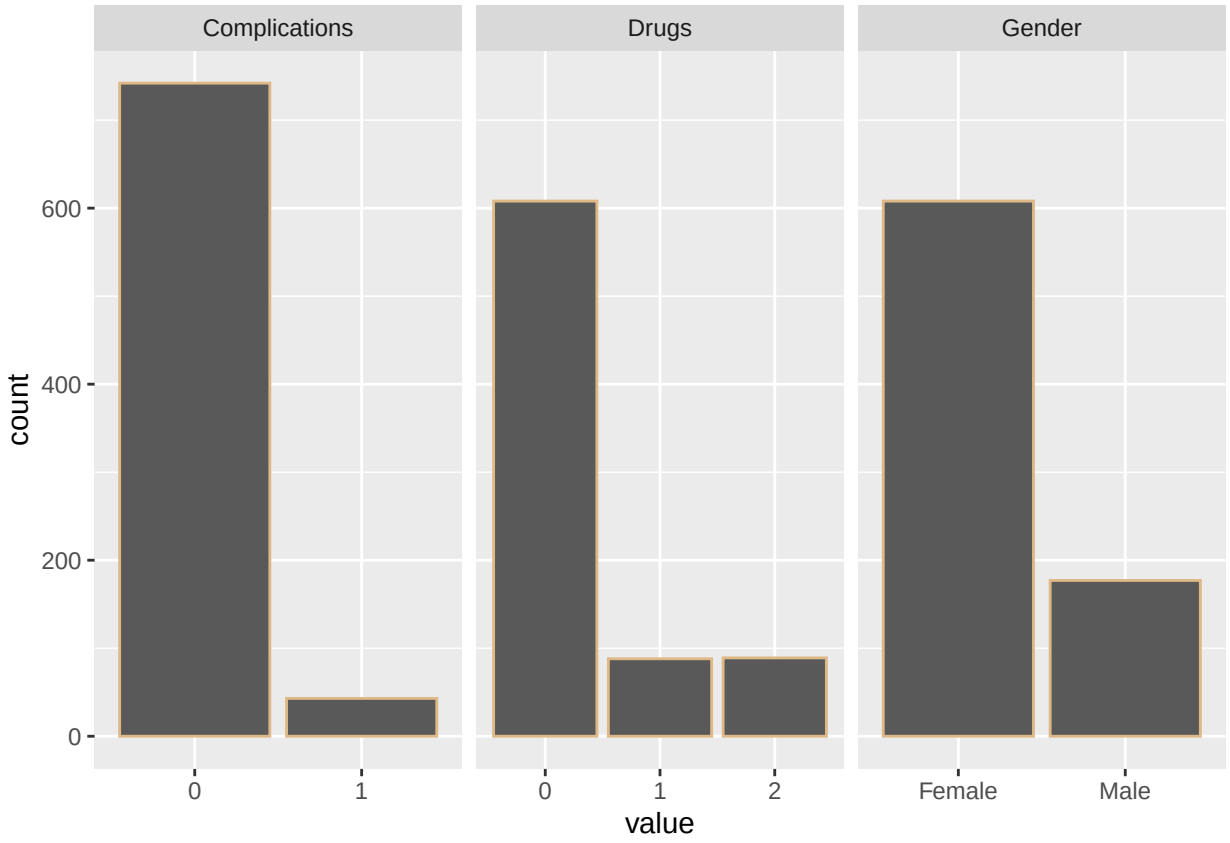
Create base-R tables and ggplot-style bar charts for `Gender`, `Drugs`, and `Complications`. Employ faceting for the bar charts! (See today's notes for guidance on how to create bar charts, while noting that, as above, the `x` value in the call to `aes()` is `value`.)

(Ignore any warnings about "attributes are not identical…")

```
selected<-df%>%select(Gender, Drugs, Complications)
df.qual <-selected%>%gather(.)
```

```
## Warning: attributes are not identical across measure variables; they will be
## dropped
```

```
ggplot(data = df.qual, mapping = aes(x = value)) + geom_bar(col = "burlywood") + facet_wrap(~key, scales
```



Question 8

Let's visualize `Drugs` and `Complications` at the same time. One way to do this is via a two-way table: simply pass both variable names to `table()` and see what happens. Such visualization can also be done in `ggplot` but it's a more complicated task than we really need to tackle here.

```
table(df$Drugs, df$Complications)
```

```
##
##        0   1
##    0 580  28
##    1  81   7
##    2  81   8
```

Question 9

Let's assume that `Cost` is our response variable: ultimately we want to learn *regression models* that predict `Cost` given the values of the remaining (predictor) variables. (We'll actually do this later!) What we might want to do now is see how `Cost` varies as a function of other variables.

First job: create side-by-side boxplots for `Cost` vs. `Gender`, `Cost` vs. `Drugs`, and `Cost` vs. `Complications`. As you might guess, we can do this via faceting, but it's a little more complicated. Just a little. Above, in the calls to `aes()`, the argument was `x = value`. Now that we have another variable, we need to add

another argument: use `y = rep(df$Cost, 3)`. This repeats the response vector three times, one time for each of the categorical predictors.

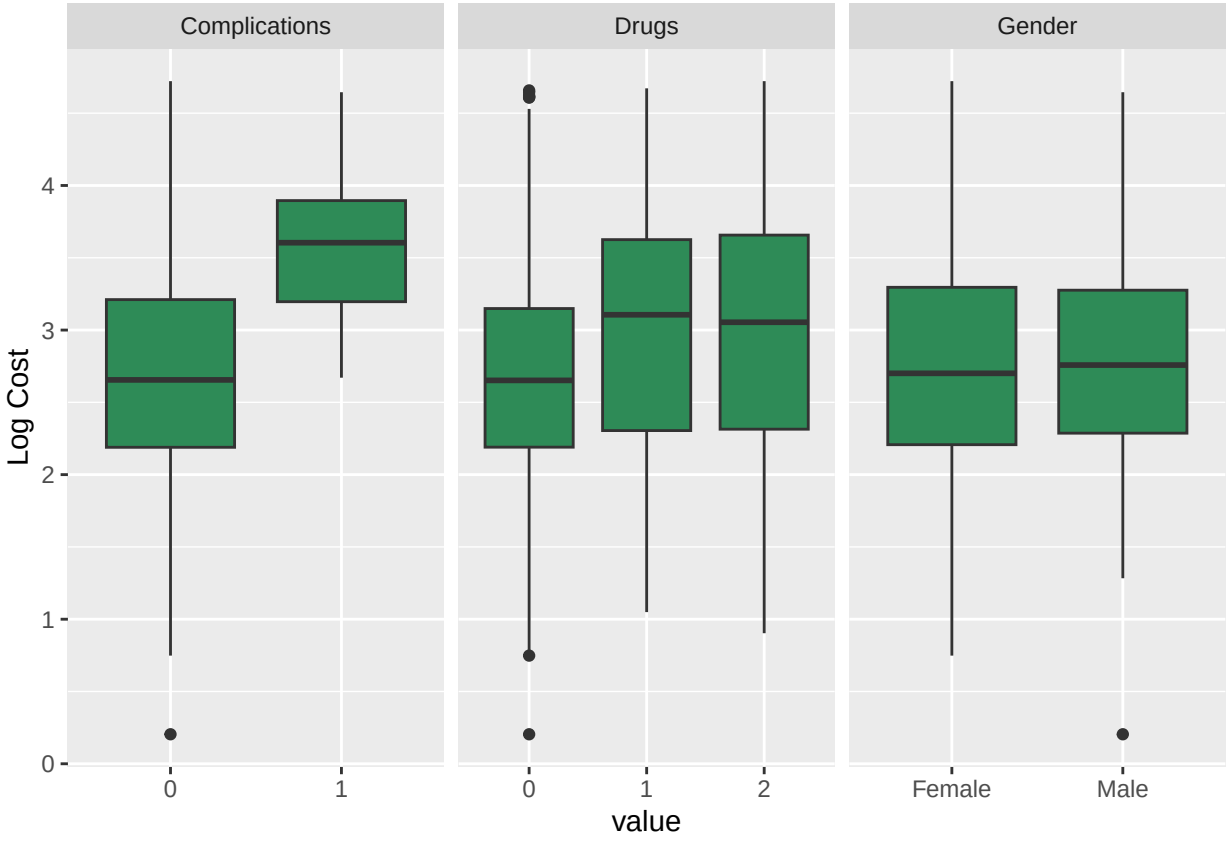
To make the y-axis label look better, add on the following after `facet_wrap()`: `+ ylab("Log Cost")`.

Do any of the categorical variables appear to be associated with the values of the response variable? (No need to write down an answer – just look at the plots and come to a conclusion in your mind.)

```
selected<-df%>%select(Gender, Drugs, Complications, Cost)
df.qual <-selected%>%gather(key = "Variable", value = "value", -Cost)
```

```
## Warning: attributes are not identical across measure variables; they will be
## dropped
```

```
ggplot(data = df.qual, mapping = aes(x = value, y = Cost)) +
  geom_boxplot(fill = "seagreen") +
        facet_wrap(~Variable, scales = "free_x") + ylab("Log Cost")
```



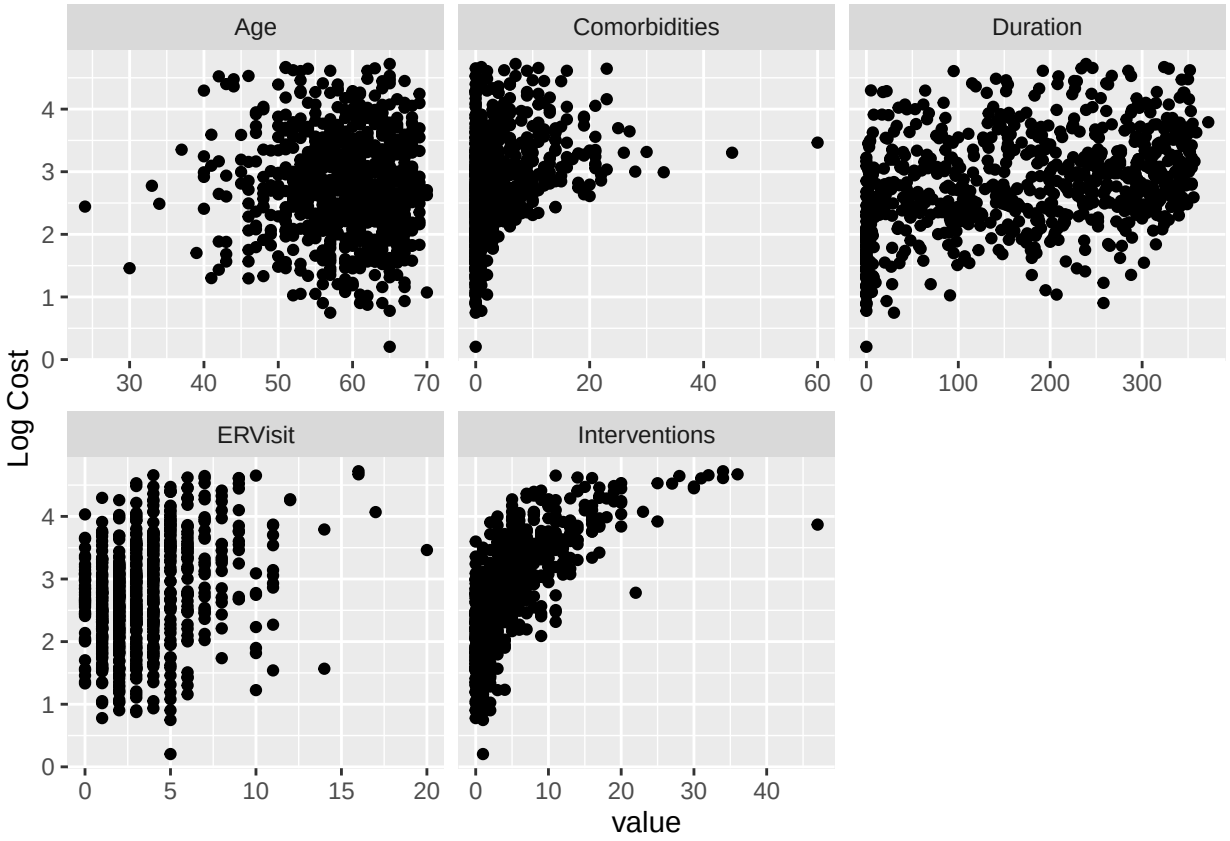
Question 10

Your next job: show scatter plots of `Cost` (on the y-axis) versus each of the remaining predictor variables. As for faceting...do what you did in Question 9, but replace `geom_boxplot()` with `geom_point()`.

Note: when using `select()`, make sure to take `Cost` out! We don't need to plot `Cost` vs. `Cost` here. Also, there are five quantitative predictor variables.

Which quantitative predictors appear to be associated with the response variable? (Again, answer this to yourself, mentally.)

```
selected<-df%>%select(-Gender, -Drugs, -Complications)
df.quant <-selected%>%gather(key = "Variable", value = "value", -Cost)
ggplot(data = df.quant, mapping = aes(x = value, y = Cost)) +
  geom_point(fill = "seagreen") +
        facet_wrap(~Variable, scales = "free_x") + ylab("Log Cost")
```



Question 11

And your last task: visually determine the level of correlation (i.e., a quantification of the strength of linear dependence) between all the quantitative predictor variables. We didn't talk about this in class, so:

First, install the package `corrplot`. Once that is done, uncomment the line `suppressMessages(library(corrplot))` below.

Then, select the quantitative predictors (again, don't select `Cost`!) and pass the output from `select()` into `cor()` (no arguments, save for the "."), and pass the output from `cor()` into `corrplot()` (with first argument ".", and second argument `method = "<something>"`; take a look at the documentation for the choices of `method` – I like "ellipse" as my `"<something>"`).

Now consider: why might apparent associations between predictor variables be bad, if you see any? We'll talk about this at length later in the course, but in short, it would be evidence of *multicollinearity*, a phenomenon which can affect your ability to interpret models that you learn, as well as generate useful statistical inferences about quantities inherent to those models (*in particular*, linear regression models).

```r
#install.packages("corrplot")
suppressMessages(library(corrplot))
```

```
## Warning: package 'corrplot' was built under R version 4.4.3
```

```r
df.quant<- df%>%select(-Gender, -Drugs, -Complications)
matrix<-cor(df.quant)
corrplot(matrix, method = "ellipse")
```